

# Uncertainty Quantification for Adaptive Measurement Noise in Kalman Filter based Object Tracking

1<sup>st</sup> Fabian Ulreich

CARISSMA Institute of Automated Driving (C-IAD)  
Technische Hochschule Ingolstadt  
85049 Ingolstadt, Germany  
fabian.ulreich@carissma.eu

2<sup>nd</sup> André Kaup

Chair of Multimedia Communications and Signal Processing  
Friedrich-Alexander-Universität Erlangen-Nürnberg  
91058 Erlangen, Germany  
andre.kaup@fau.de

3<sup>rd</sup> Martin Ebert

Faculty of Electrical Engineering and Information Technology  
Technische Hochschule Ingolstadt  
85049 Ingolstadt, Germany  
martin.ebert@thi.de

**Abstract**—The application of deep learning systems in the perception of autonomous vehicles results in the need for information about the trust in its predictions. In our work, we use methods from the field of uncertainty quantification and probability theory to natively integrate the 3D object detection stage into multi-object tracking for perception. For this purpose, we fine-tune a state-of-the-art sensor fusion model for camera and LiDAR data to output a full probability distribution. We propose a calibration scheme and we quantitatively show that predictive uncertainty can be linked to object occlusion. We furthermore compare predictive uncertainty with post-hoc calculated empirical uncertainty in terms of calibration and tracking performance. Through our approach, we reduce the calibration error of the center  $x$ -coordinates about 92% and of the center  $y$ -coordinates about 87%. Moreover, we utilize the predictive uncertainty estimate of our detector as the measurement variance in Kalman filtering for multi-object tracking, reducing average tracking position error on the nuScenes data set by about 1.3%.

**Index Terms**—Object tracking, uncertainty quantification, calibration, sensor fusion, Kalman filter, deep learning

## I. INTRODUCTION

In an automated vehicle (AV) the 3D-object detection of a perception system processes sensor signals to estimate geometrical properties like position and velocity, but also semantical attributes of objects relevant to the overall driving task. The output of the detection stage can be further processed by a tracking stage to identify objects over multiple time frames or to predict future behavior as part of a larger perception system.

Because deep learning (DL) systems tend to be overconfident [1], the probability yielded by a DL model does not necessarily match the actual probability. It is also often neglected in further processing steps of perception pipelines, e.g. the Autoware<sup>1</sup> camera and LiDAR perception, which only apply a score threshold filter on the detections. Frequently,

DL models are used as 3D object detectors and thereby they suffer from this behavior as well. This is the subject of the field of uncertainty quantification (UQ) which covers methods to express trust in the predictions of a DL system, aiming to yield calibrated probabilities instead of single-point estimates.

With this paper, we want to bridge between the fields of perception and UQ with an application on AVs, to inherently increase the robustness and safety of such systems. We also show how UQ establishes a way to propagate trust in the predictions of a DL forecaster to a downstream task, namely multi-object tracking, using probability theory. For this, we base our work on a state-of-the-art fusion model [2], which processes images from multiple cameras and an LiDAR. We fine-tune the model on the nuScenes data set [3] to output a full probability distribution on its predictions. Additionally, we extensively focus on the task of calibration to increase the validity of the probabilities and empirically show the expressiveness in the center  $y$ - and  $x$ -coordinates under object occlusion. Afterward, we couple the detection and the tracking step via predictive uncertainty, by conditioning the measurement covariance used in the update step in Kalman filtering on the sensor data. Within the work, we propose an alternative way towards safe operation by system design, in contrast to approaches focusing on validation techniques like test-benches and simulation [4], [5].

## II. RELATED WORK

### A. Multi-object Tracking

The goal in (online) multi-object-tracking (MOT) via track-by-detection is to compute the trajectory of an object, sequentially from the past to the current time-step, based on observations. One technique is to use Kalman filtering (KF) to recursively filter observations under the assumption of Gaussian measurement and process noise [6]–[9]. In both cases of

<sup>1</sup><https://github.com/autowarefoundation/autoware.universe>

linear and non-linear measurement models, the belief at time step  $k$  about the state  $\mathbf{z}_k$  is updated with the observations  $\mathbf{m}_k$ . For this purpose, a Gaussian distribution on the measurements  $\mathbf{m}_k$  is assumed to be additive and with zero mean.

$$\mathbf{m}_k = \mathbf{H}\mathbf{z}_k + \mathbf{r}_k, \text{ where } \mathbf{r}_k \sim \mathcal{N}(0, \mathbf{R}), \quad (1)$$

$$p(\mathbf{m}_k | \mathbf{z}_k) = \mathcal{N}(\mathbf{m}_k | \mathbf{H}\mathbf{z}_k, \mathbf{R}) \quad (2)$$

KF treats (1) as a different representation of (2). The specific choice of the state depends on the motion model used (e.g. constant velocity model). The measurement matrix  $\mathbf{H}$  ( $N \times M$ ) links the observations (dimension  $M$ ) with the states (dimension  $N$ ). Within the scope of this article,  $\mathbf{y}_k$  corresponds to the noise-free mapping of the state vector into the measurement space.

$$\mathbf{y}_k = \mathbf{H}\mathbf{z}_k \quad (3)$$

Trackers utilizing KF often treat the measurement covariance  $\mathbf{R}$  as a hyperparameter, which is optimized on a split of the data set [6]. The authors of [9] in contrast, calculate the measurement covariance on the training data set according to (5).

$$\delta = \mathbf{m}_k - \mathbf{y}_{k,pred} \quad (4)$$

$$\mathbf{R} = \text{Var}[\delta] = \sigma_{\text{emp}}^2 \quad (5)$$

With this work we move one step further and use the full probability distribution of our detector to condition the measurement on the sensor data of the current time step  $k$  and instead optimize calibration on the training data (see Fig. 1 C)).

### B. Uncertainty Quantification and Calibration

Uncertainty quantification deals with methods and techniques to capture the uncertainty in the predictions made by a forecaster  $\mathcal{H}$  defined by its set of weights  $\mathbf{w}$  [10]–[12]. Two major sources of uncertainty exist: First, there is epistemic uncertainty which can be reduced e.g. by selecting a different model architecture (inductive bias) or adding training data. Secondly, the aleatoric uncertainty is not reducible, as it rises from the randomness of the data-generating process itself. Uncertainty treated from the "Bayesian" perspective can be expressed naturally by formulating the prediction of a forecaster  $\mathcal{H}$  as a probability density function  $p$ . Given a data set with its samples  $\mathcal{D} = \{\mathbf{x}_i, \mathbf{y}_i\}$   $\mathcal{H}$  outputs:

$$p(\mathbf{y}|\mathbf{x}, \mathcal{D}) = \int p(\mathbf{y} | \mathbf{x}\mathbf{w}, D)p(\mathbf{w} | \mathcal{D})d\mathbf{w} \quad (6)$$

This marginal distribution contains both epistemic,  $p(\mathbf{w} | \mathcal{D})$  and aleatoric uncertainty,  $p(\mathbf{y} | \mathbf{w}, D)$ . Furthermore, aleatoric uncertainty is modeled in two ways: Homoscedastic uncertainty and heteroscedastic uncertainty [12]. The former treats uncertainty constant and the latter estimates data noise for each sample. We label  $F^{-1}(p)$  as the inverse of the cumulative distribution function (CDF) outputted by the forecaster,

following the notation of [13], which extends the works of [1] on classification. In regression,  $\mathcal{H}$  is calibrated, if:

$$\frac{\sum_{i=1}^T \mathbb{1}\{\mathbf{y}_i \leq F_i^{-1}(p)\}}{T} \rightarrow p, \forall p \in [0, 1] \quad (7)$$

with  $\mathbb{1}$ , being the indicator function and  $T$  corresponding to the size of the data set. (7) expresses calibration as a match between the probability  $p$  of target values falling in the confidence interval given by  $p$ .

In [12], Monte-Carlo dropout is used together with Gaussian negative log-likelihood loss to estimate the predictive uncertainty for the task of monocular depth estimation and semantic segmentation with CNNs. Landgraf et al. suggest an extension in [14], in which they use the technique in vision transformers and leverage UQ for a student-teacher distillation approach. In their approach, they base epistemic UQ on a deep-sub-ensemble approach, following [15].

Additionally, Feng et al. [16] apply heteroscedastic aleatoric uncertainty quantification by utilizing Gaussian negative log-likelihood loss to a LiDAR 3D object detector, using multiple, vertically sliced birds-eye view (BEV) maps as network inputs. Afterward, they relate aleatoric uncertainty to object occlusion and distance in a qualitative study, showing expressiveness at individual examples. In [17] the authors further extend their work and evaluate different calibration techniques, namely temperature scaling and isotonic regression on the previously developed detector. However their approach only captures aleatoric uncertainty. Furthermore, they propose using a calibration loss during training and experiment with different calibration technique combinations on a subset of the KITTI validation data set.

In [18] the approach of temperature scaling [1] is also used for the task of object detection. In contrast to isotonic regression, approaches following temperature scaling require an optimization technique, as they are based on minimizing the log-likelihood.

Overall, we are the first to conduct experiments bringing uncertainty quantification and following task like 3D object tracking together. In current approaches, the measurement covariance  $\mathbf{R}$  is computed statistically by evaluating  $\mathcal{H}$  on training data, but in our case,  $\mathbf{R}$  will be conditioned on the sensor data (camera, LiDAR), as the confidence taken on the measurement should be dependent on the current driving situation itself, and not only influenced by the training data set. This can be emphasized as the training data domain, expressed as the joint distribution of samples and targets  $p_{\text{data}}(\mathbf{x}, \mathbf{y})$ , does not necessarily have to be equal to the distribution in specific situations under operation.

## III. METHODS

Next, we introduce how we extend previous works on MOT by taking the possibilities of UQ into account. We define our subject of research to be a perception system of an autonomous vehicle that tracks the trajectories of traffic participants in the surroundings of the ego vehicle.

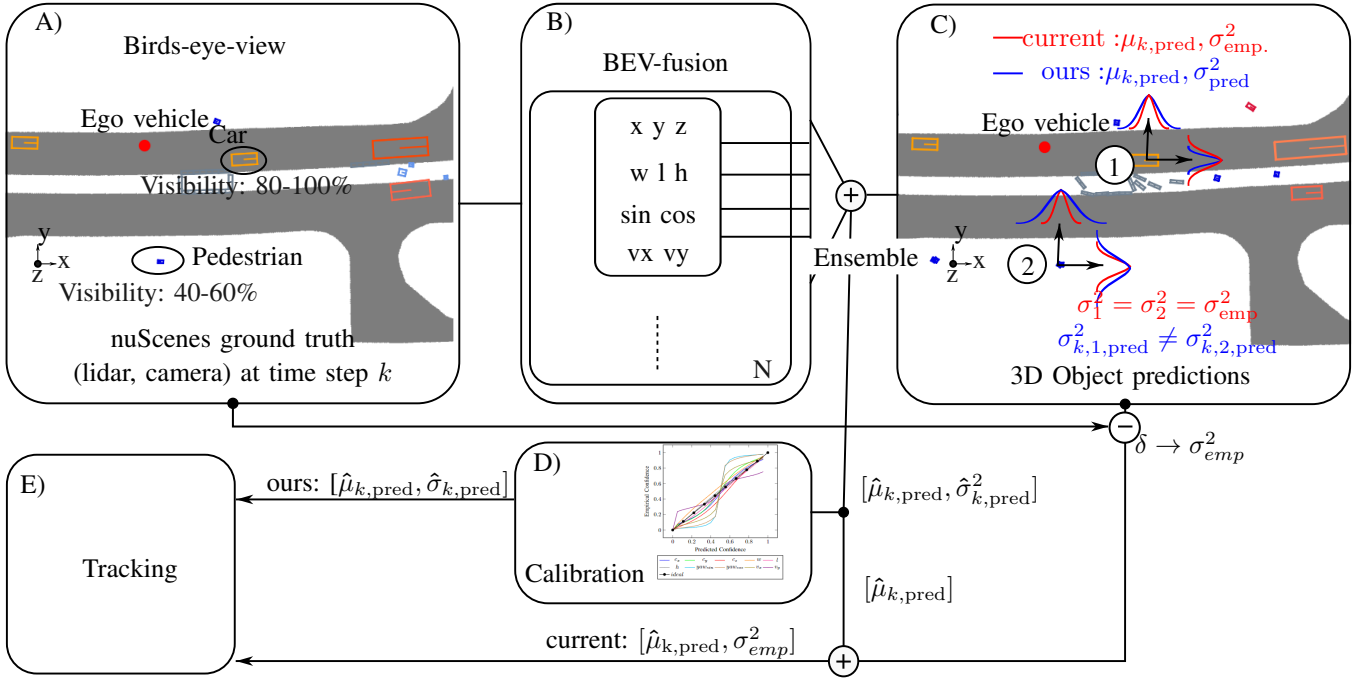


Fig. 1: A): NuScenes data set [3] annotations visualized in BEV with the corresponding visibility of the object. B): BEV-fusion network [2], operating on the images of six cameras and LiDAR data with our modifications incorporating the uncertainty head and sub-ensemble, fused predictions of ensembles showing the predicted distribution according to (10) in  $x$ - and  $y$ -direction (C)). In current KF-based tracking approaches  $\sigma_{emp}^2$  is selected as the measurement covariance, in contrast to our approach utilizing predictive uncertainty after calibration,  $\hat{\sigma}_{pred,calib}^2$ . The motivation behind this approach can be seen in C). Predictive uncertainty is larger for objects with low visibility and different in  $x$ - and  $y$ -direction based on the orientation of the object with respect to the ego-vehicle (compare 1 and 2), in contrast to current approaches. To ensure that predictive uncertainty is calibrated, according to (7), we performed a calibration procedure in step D). E): Two different approaches in tracking, either utilizing object detectors predictions  $[\hat{\mu}_{k,pred}, \hat{\sigma}_{k,pred}^2]$  like we propose, or following the state of the art  $[\hat{\mu}_{k,pred}, \sigma_{emp}^2]$ .

For this task, a 3D detector processes samples of the nuScenes data set, consisting of a single LiDAR and multiple camera images, and feeds its predictions consisting of the objects class label  $c$ , its probability and the geometrical properties of the object to the following tracking module (see III-C), based on Kalman filtering (see Fig. 1).

Fig. 1 emphasizes the challenges in the surroundings of an AV. The length and the center  $y$ -coordinate of the vehicle right to the ego vehicle (highlighted with the circle) are expected to have higher aleatoric uncertainty, and predictive uncertainty as well, as compared to the width and the center  $x$ -coordinate. The assumption behind this is, that due to the orientation of the vehicle, we do not directly acquire measurements of the length and the center  $y$ -coordinate with the sensors of the ego vehicle, thereby inducing randomness. We assume, because of similar effects, that objects with lower visibility should have higher aleatoric and predictive uncertainty in general.

#### A. Uncertainty Quantification in Transformer based BEV-fusion

In order to capture aleatoric uncertainty, we need to reformulate the training objective of an existing 3D object detector by adjusting the loss function. We base our method

on the architecture of BEV-fusion [2] which consists of a Vision Transformer decoder operating on BEV-space for 3D object detection. The selected architecture has two individual encoder branches for LiDAR, and camera respectively. The computed encoding of both branches is fused by a CNN. The dimensional output of the fused embeddings matches already the spatial dimensions of the BEV grid which is afterward used for query generation in the following multi-layer Transformer decoder (see Fig. 1). We base our modification on the loss function in object detection consisting of two major parts:

$$\mathcal{L} = \mathcal{L}_{\text{classification}} + \mathcal{L}_{\text{regression}} \quad (8)$$

In  $\mathcal{L}_{\text{regression}}$  we change the original implementation of BEV-fusion and employ the Gaussian negative log likelihood loss, which calculates the likelihood of the data  $\mathbf{y}_k = [y_{c_x}, y_{c_y}, y_{c_z}, y_w, y_l, y_h, y_{\sin}, y_{\cos}, y_{v_x}, y_{v_y}]^T$ , based on a Gaussian probability distribution parameterized by the mean  $\mu_{k,pred}$  and the variance  $\sigma_{k,pred}^2$ . The targets, learned by our 3D decoder network, are  $[\mu_{c_x}, \mu_{c_y}, \mu_{c_z}, \mu_w, \mu_l, \mu_h, \mu_{\sin}, \mu_{\cos}, \mu_{v_x}, \mu_{v_y}]^T$  and  $[\sigma_{c_x}^2, \sigma_{c_y}^2, \sigma_{c_z}^2, \sigma_w^2, \sigma_l^2, \sigma_h^2, \sigma_{\sin}^2, \sigma_{\cos}^2, \sigma_{v_x}^2, \sigma_{v_y}^2]^T$ . Based on this choice of loss function, our forecaster outputs a full

probability distribution, at time step  $k$  with heteroscedastic noise:

$$p(\mathbf{y}_k | \mathbf{x}_k, \mathcal{D}) = \mathcal{N}(\mathbf{y}_k | \mu_{k,\text{pred}}, \sigma_{k,\text{pred}}^2) \quad (9)$$

Our solution has no separate branch to estimate  $\sigma_{k,\text{pred}}^2$ , compared to [12], [14]. Instead, we perform aleatoric uncertainty estimation on the MLPs of the multi-head attention module, each responsible for jointly predicting the individual  $\mu_{\text{pred}}$  and  $\sigma_{k,\text{pred}}^2$  (see Fig. 1 B)).

To capture epistemic uncertainty, we leverage the sub-ensemble approach of [15] and adjust the network architecture to have multiple parallel decoder head branches, operating on the same shared BEV space. We decide for this technique, as it only brings a small overhead concerning the number of weights, and avoids also the need to sample during inference time as required by approaches following Monte-Carlo dropout [12]. During training, we sample from a uniform distribution on each batch, to select which transformer head will be optimized. The ensemble can be afterward grouped easily, as the query generation for the individual transformer decoder heads is based on the same shared BEV space heatmap. The predictive distribution is obtained by the ensemble of the predictions of the individual decoder branches element-wise [19].

$$p(\mathbf{y}_k | \mathbf{x}_k, \mathcal{D}) \approx \mathcal{N}(\mathbf{y}_k | \hat{\mu}_{k,\text{pred}}, \hat{\sigma}_{k,\text{pred}}^2), \text{ where} \quad (10)$$

$$\hat{\mu}_{k,\text{pred}} = \frac{1}{N} \sum_{i=0}^N \mu_{k,i,\text{pred}} \quad (11)$$

$$\hat{\sigma}_{k,\text{pred}}^2 = \frac{1}{N} \sum_{i=0}^N \sigma_{k,i,\text{pred}}^2 + \frac{1}{N} \sum_{i=0}^N (\mu_{k,i,\text{pred}} - \hat{\mu}_{k,\text{pred}})^2 \quad (12)$$

### B. Calibration of Uncertainty

To improve the calibration of the regression outputs of our networks, we create an auxiliary branch in the model, which is only active during calibration training hooks. It is trained by minimizing the log-likelihood following the approach of temperature scaling. In correspondence to the  $N$  individual decoder branches, each is assigned a calibration layer consisting of an MLP taking  $\mu_{k,\text{pred},c_x}$  and  $\mu_{k,\text{pred},c_y}$  and the variances  $\sigma_{k,\text{pred}}^2$  as inputs. By explicit conditioning on the predicted object center coordinates, we ground aleatoric uncertainty calibration mainly on the distance and yaw angle of the object with respect to the ego vehicle. We select the Expected Calibration Error (*ECE*) according to [13] to measure calibration performance.

$$ECE = \sum_{i=m}^N w_i (p_m - \hat{p}_m)^2 \quad (13)$$

The *ECE* captures the mismatch between the data distribution  $p_{\text{data}}(\mathbf{y}, \mathbf{x})$  and the (ensemble) predictions of the forecaster, calculated by the squared difference between the expected confidence and the empirical frequency given by (7). In our

case, we choose the weight  $w_i$  to be equal for each confidence bin.

Additionally, we scale ensembles predictive uncertainty  $\hat{\sigma}_{k,\text{pred}}^2$  that their expected value given the training data,

$$E_{\text{data}}[\lambda_c \hat{\sigma}_{k,\text{pred}}^2] = \sigma_{\text{emp},c}^2 \quad (14)$$

equals the empirical variance  $\sigma_{\text{emp}}^2$ , given by (5). It should be noted, that calculation of the scaling factor  $\lambda_c$  happens for each class separately.

### C. Coupling Detection with Tracking via Uncertainty

In our track-by-detection setup the measurement noise  $\mathbf{R}$  not only depends on the sensor characteristics of the ego vehicle but is affected by the characteristics of the detector, as well. We furthermore categorize those two sources of noise into aleatoric (sensor noise) and epistemic (model noise).

Our idea is to link the distribution given by (9) to the update step in KF-based on (1). By assuming that our forecaster  $\mathcal{H}$  outputs an unbiased estimate about  $\mathbf{y}_k$ , we are able to reformat (9) according to (1), by splitting the output in a deterministic part, given by  $\hat{\mu}_{k,\text{pred}}$ , and in an additive random part, expressed by the predictive uncertainty  $\hat{\sigma}_{k,\text{pred}}^2$ . Accordingly, we set the measurement covariance at time step  $k$ ,  $\mathbf{R}_k = \hat{\sigma}_{\text{pred}}^2$  after our scaling procedure (see Sec. III-B and (14)).

Thereby, we aim to achieve that the tracking system consisting of the forecaster and the tracker, can adaptively weight the predictions made on the current driving scene.

## IV. EXPERIMENTS AND RESULTS

### A. Training

We fine-tune our transformer decoder and each head for 12 epochs on the nuScenes training data split, based on the pre-trained weights of the original implementation. Moreover, we need to reduce the batch size to two (originally four), as we fine-tune the model on four RTX3090 GPUs. Following after the training epochs, we calibrate the network variance outputs for one additional epoch, using a custom hook. During calibration, we freeze the rest of the network and train only the auxiliary calibration head on a subset of the training data.

### B. Calibration

The sub-split during the calibration training procedure contains 20% of the samples. Calculation of the class-specific scale values  $\lambda_c$  instead based on the full training data. Calibration is afterward measured on the validation data set. Given our use-case of tracking, we decide to compare the class-wise scaled predictive uncertainty  $\hat{\sigma}_{\text{pred}}^2$  with the empirical uncertainty  $\sigma_{\text{emp}}^2$ , because this quantity is used in previous approaches as the measurement covariance in KF [9]. Furthermore we perform calibration as proposed by [17], [18] with temperature scaling. The results after our calibration procedure are shown in Fig. 2. Only the common sine and cosine encoded yaw angles and the velocities still deviate much from the ideal. Compared to  $v_y$  the encoded yaw angles and velocities show a s-curved calibration behavior, which can result from

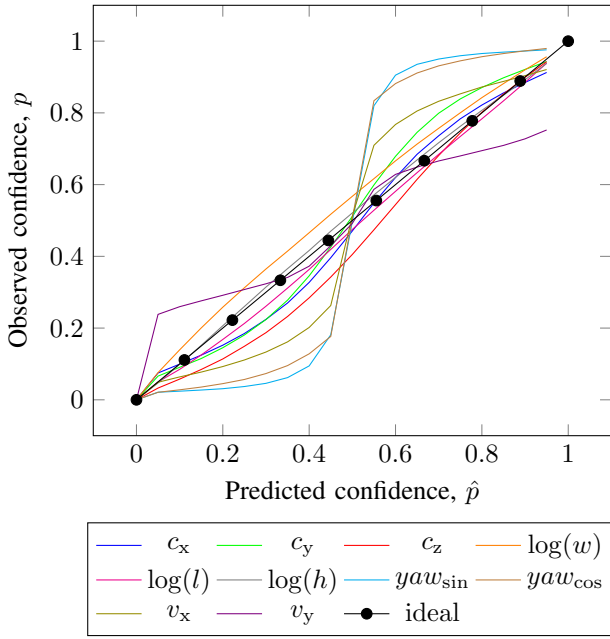


Fig. 2: Reliability diagram of the class-wise-scaled, predictive uncertainty  $\lambda_c \hat{\sigma}_{\text{pred}}^2$  after the calibration procedure. Observed confidence  $p$  is the fraction of data within  $y_t \leq F_t^{-1}(\hat{p})$  given the predicted confidence  $\hat{p}$ . Points where the observed confidence is larger than the predicted indicate under-confidence of the forecaster, in opposite points the forecaster is over-confident.

a mismatch in assumed Gaussian distribution and the actual one.

As it can be seen from Tab. I, the *ECE* is in the case of  $c_x$  92.97%, and for  $c_y$  87.01% higher compared to the proposed temperature scaling approach. Additionally we are able to reduce calibration error also compared to  $\sigma_{\text{emp}}^2$ , indicating that conditioning on sensor data can reduce the calibration error for those cases. Also, this implies that our approach is capable of extracting information from the sensor data. Additionally, also calibration is significantly better for  $v_x$  and  $v_y$ . Interestingly the temperature scaling approach reduces the calibration error for most of the network outputs compared to the predicted variance  $\hat{\sigma}_{\text{pred}}^2$ , only. But nevertheless the calibration error is higher than compared to using the empirical variance  $\sigma_{\text{emp}}^2$ , which is also obtained using training data.

### C. Linking Aleatoric Uncertainty and Object Visibility

In the next section, we further want to evaluate the expressiveness of the uncertainty estimate by linking with objects visibility on the nuScenes validation data set. Through the availability of nuScenes annotations, we assign to each matched prediction the corresponding ground truth visibility, which is discretized in the steps of 0-40%, 40-60%, 60-80%, and 80-100%. The visibility is proportional to the number of pixels the object covers in all camera frames at a time step  $k$ , compared to the size of the object. It can be seen from

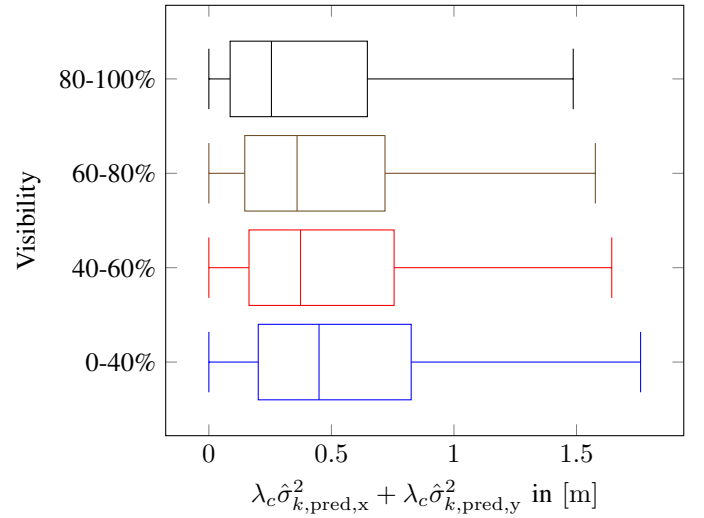


Fig. 3: Box plot showing the correlation between aggregated predictive uncertainty and the visibility of ground truth objects, each box plot shows the lower quartile  $q_{25}$ , the upper quartile  $q_{75}$ , the lower whisker  $q_{25} - 1.5(q_{75} - q_{25})$  if  $\geq 0$ , else 0, the median  $q_5$ , and the upper whisker  $q_{75} + 1.5(q_{75} - q_{25})$  in each visibility bin.

Fig. 3, as the visibility of the objects is increased, aggregated predictive uncertainty of the center  $x$ - and  $y$ -coordinates gets lower (median: 0.45; 0.37; 0.36; 0.25), indicating that our approach follows our intuition (highlighted in Fig. 1 up-left and up-right) and captures situation-awareness, based on object occlusion. Moreover, predictive uncertainty has some variance within each of the visibility classes. In conclusion, our approach is not only able to yield calibrated estimates, but also links predictive uncertainty with properties of the current driving scenario, like the occlusion of objects.

### D. Evaluation on NuScenes

With respect to the task of tracking, we evaluate the PolyMOT tracker [6] and a second KF-based Tracker [9] with measurement covariance  $R_k = \lambda_c \hat{\sigma}_{\text{pred}}^2$ , to adaptive weight the measurement confidence, based on the current sensory input to the car.

During the evaluation, we perform three (two) types of experiments, each with the original parameterization of the tracker [6], using empirical variance as measurement covariance ( $R_k = \sigma_{\text{emp}}^2$ ) and finally our proposed solution. In the case of [9] the measurement covariance is already set to be equal to  $\sigma_{\text{emp}}^2$ . Additionally, in order to omit non-Gaussian properties of the sine- and cosine-encoded yaw-angle and the log of the width, height, and length of the object, we use the empirical variance in all of the cases. To perform the experiment, we convert predicted variances in the LiDAR-coordinate frame to the global coordinate frame with the corresponding rotation matrices. The first experiment is evaluated on the complete nuScenes validation data set. The second and third experiments are both carried out on a sub-

TABLE I: Comparison between  $ECE \downarrow$  for the class car, using the full predicted distribution, given by (10) and  $p(\mathbf{y}_k | \mathbf{x}_k, \mathcal{D}) = \mathcal{N}(\mathbf{y} | \hat{\mu}_{\text{pred}}, \sigma_{\text{emp}}^2)$ , in which  $\sigma_{\text{emp}}^2$  is obtained using training data.

| Quantity     | $ECE$ from [18] on training data | $ECE$ before scaling $\hat{\sigma}_{\text{pred}}^2$ | $ECE$ with $\sigma_{\text{emp}}^2$ | $ECE$ with $\lambda_c \hat{\sigma}_{\text{pred}}^2$ | improvement |
|--------------|----------------------------------|-----------------------------------------------------|------------------------------------|-----------------------------------------------------|-------------|
| $c_x$        | 1.309                            | 1.317                                               | 0.489                              | <b>0.092</b>                                        | -92.97%     |
| $c_y$        | 1.34                             | 1.351                                               | 0.277                              | <b>0.174</b>                                        | -87.01%     |
| $c_z$        | 0.573                            | 1.129                                               | <b>0.097</b>                       | 0.237                                               | -58.64%     |
| $\log(w)$    | 1.427                            | 1.26                                                | 0.225                              | <b>0.133</b>                                        | -90.68%     |
| $\log(l)$    | 1.422                            | 1.417                                               | <b>0.71</b>                        | 0.793                                               | -44.23%     |
| $\log(h)$    | 0.934                            | 1.144                                               | 0.164                              | <b>0.007</b>                                        | -99.25%     |
| $yaw_{\sin}$ | 1.028                            | 0.933                                               | 1.099                              | <b>0.867</b>                                        | -15.66%     |
| $yaw_{\cos}$ | 1.102                            | 0.887                                               | 1.117                              | <b>0.749</b>                                        | -32.03%     |
| $v_x$        | 0.831                            | 0.988                                               | 0.86                               | <b>0.313</b>                                        | -62.33%     |
| $v_y$        | 0.858                            | 0.982                                               | 0.606                              | <b>0.171</b>                                        | -80.07%     |

TABLE II: Tracking results on the nuScenes data set, a subsplit with average scene ground truth (gt) visibility greater than two, or less or equal to two. The model column distinguishes between original Poly-MOT [6] parameterization, measurement covariance either set to  $\lambda_c \hat{\sigma}_{\text{pred}}^2$  or  $\hat{\sigma}_{\text{emp}}^2$  and the implementation of [9] already using  $\sigma_{\text{emp}}^2$  and our change to use  $\lambda_c \hat{\sigma}_{\text{pred}}^2$ .

| Model                                                | Data set                    | Amota [0,1] $\uparrow$ | Amotp [0, $\infty$ ] in [cm] $\downarrow$ | Recall [0,1] $\uparrow$ |
|------------------------------------------------------|-----------------------------|------------------------|-------------------------------------------|-------------------------|
| reference acc. to [6]                                | complete                    | <b>72.00</b>           | 66.94                                     | 76.25                   |
| $R_k = \lambda_c \hat{\sigma}_{k,\text{pred}}^2$     | complete                    | 71.81                  | <b>66.07</b>                              | <b>76.86</b>            |
| $R_k = \sigma_{\text{emp}}^2$                        | complete                    | 71.99                  | 66.08                                     | 76.49                   |
| <b>ours vs. reference</b>                            | complete                    | -0.26%                 | -1.3%                                     | 0.8%                    |
| reference acc. to [6]                                | avg. visibility $\leq 60\%$ | 68.32                  | 72.23                                     | <b>71.47</b>            |
| $R_k = \lambda_c \hat{\sigma}_{k,\text{pred}}^2$     | avg. visibility $\leq 60\%$ | <b>68.37</b>           | 72.07                                     | 72.15                   |
| $R_k = \sigma_{\text{emp}}^2$                        | avg. visibility $\leq 60\%$ | 68.22                  | <b>71.7</b>                               | 71.39                   |
| <b>ours vs. reference</b>                            | avg. visibility $\leq 60\%$ | 0.07%                  | -0.22%                                    | 0.95%                   |
| reference acc. to [6]                                | avg. visibility $> 60\%$    | <b>73.00</b>           | 65.30                                     | 76.81                   |
| $R_k = \lambda_c \hat{\sigma}_{k,\text{pred}}^2$     | avg. visibility $> 60\%$    | 72.82                  | <b>64.37</b>                              | 76.81                   |
| $R_k = \sigma_{\text{emp}}^2$                        | avg. visibility $> 60\%$    | 72.95                  | 64.39                                     | <b>77.02</b>            |
| <b>ours vs. reference</b>                            | avg. visibility $> 60\%$    | -0.25%                 | -1.42%                                    | 0%                      |
| reference acc. to [9], $R_k = \sigma_{\text{emp}}^2$ | complete                    | <b>63.80</b>           | 70.09                                     | 69.36                   |
| $R_k = \lambda_c \hat{\sigma}_{k,\text{pred}}^2$     | complete                    | 63.75                  | <b>69.61</b>                              | <b>69.82</b>            |
| <b>ours vs. reference</b>                            | complete                    | -0.08 %                | -0.68%                                    | 0.66%                   |
| reference acc. to [9], $R_k = \sigma_{\text{emp}}^2$ | avg. visibility $\leq 60\%$ | 61.63                  | 74.64                                     | <b>67.21</b>            |
| $R_k = \lambda_c \hat{\sigma}_{k,\text{pred}}^2$     | avg. visibility $\leq 60\%$ | <b>62.07</b>           | <b>73.85</b>                              | 67.05                   |
| <b>ours vs. reference</b>                            | avg. visibility $\leq 60\%$ | 0.71%                  | -1.06%                                    | -0.24%                  |
| acc. to [9] $R_k = \sigma_{\text{emp}}^2$            | avg. visibility $> 60\%$    | <b>64.41</b>           | <b>68.44</b>                              | 70.37                   |
| $R_k = \lambda_c \hat{\sigma}_{k,\text{pred}}^2$     | avg. visibility $> 60\%$    | 64.13                  | 68.59                                     | <b>71.19</b>            |
| <b>ours vs. reference</b>                            | avg. visibility $> 60\%$    | -0.43%                 | 0.22%                                     | 1.17%                   |

split of the nuScenes validation data set. For that purpose, we first compute the average of the ground truth visibility of the object within one scene, and afterward separate the data set in two splits, either with average ground truth visibility larger than two, or less or equal two. The low-visibility data set contains 1204 scenes and the high-visibility 4815. We find the separation with respect to scenes, because our 3D detector used multiple-LiDAR sweeps in one scene. For comparison, we leverage the AMOTA and AMOTP metric of the official nuScenes tracking challenge [3]. In general, the nuScenes AMOTA metric is based on the definition according to [8],

but further takes the recall into account:

$$MOTA_r = \max(0, 1 - \frac{IDS_r + FP_r + FN_r - (1-r)P}{rP}) \quad (15)$$

$$AMOTA = \frac{1}{|R|} \sum_{r \in R} MOTA_r. \quad (16)$$

According to this definition AMOTA captures the number of erroneous detections as the sum of identity switches ( $IDS$ ), the false positives ( $FP$ ), the false negatives ( $FN$ ) at a specific recall level. Additionally, the class-specific number of ground

truth positives  $P$  is used. For the AMOTP metric the authors follow [20]:

$$MOTP = \frac{1}{|R|} \sum_{r \in R} \frac{\sum_{i,t,r} d_{i,t}}{\sum_t TP_{t,r}} \quad (17)$$

With  $d_{i,t}$  being the position error of the  $i$ -th trajectory at time step  $t$  and  $TP_t$  the number of ground truth at this time step. Note that averaging is afterward performed on the recall levels  $r$ .

The results are aggregated in Tab. II. We compare the tracking results of our modification in [6]. Interestingly, utilizing either empirical variance or predictive uncertainty has the biggest impact on the tracking precision, with reductions of 1.3% on average on the complete data set, 0.22% on the low-visibility and 1.42% on the high-visibility data set when using predictive uncertainty. The tracking accuracy instead is almost not influenced. Nevertheless, also recall can be increased in all of the cases, and we not only achieve more precise but also a better ratio between  $TP$  and  $FN$ , indicating that potentially  $IDS$  or the tracking score result in decrease tracking accuracy. The same holds after utilizing predictive uncertainty in the case of [9], where we achieve a 1.06% improvement in the low-visibility data set and 0.68% on the complete data set in terms of AMOTP. In conclusion, we can state that our approach is able to improve the tracking precision, especially in challenging scenarios, where the visibility of objects is low, despite its only small impact on the tracking accuracy. Also we can highlight that in the case of [9] as the reference tracker the improvement in using predictive uncertainty compared to using empirical variance, is even higher than in the case of [6].

## V. CONCLUSION AND OUTLOOK

Within this paper, we present a way to tightly couple the detection step with the tracking step in the perception of an AV. With our approach we are able to show that the average position error of the tracked trajectories, measured by the AMOTP can be reduced on the nuScenes data set by about 1.06% for the case of [9]. This already indicates that conditioning measurement noise on sensor data gives improvements and taking full predictive uncertainty into account might further improve the results, as we leverage only center coordinates predictive uncertainties. However, the tracking improvements are less than the improvements determined in terms of  $ECE$ , indicating that other metrics are required to improve calibration for the task of Kalman filtering in multi-object tracking. Also, we like to highlight, that these are average metrics and further investigation in challenging corner cases, which might lead to dangerous hazards, need to be performed. Additionally, we leave classification calibration for further investigation, as we expect further improvement in terms of tracking accuracy.

## ACKNOWLEDGMENT

The authors would like to thank Dominik Bayerl for the support and Prof. Dr.-Ing Joachim Hof from the CARISMA

Institute of Electric, Connected and Secure Mobility for providing access to the computing infrastructure.

## REFERENCES

- [1] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, "On Calibration of Modern Neural Networks," Aug. 2017, arXiv:1706.04599 [cs].
- [2] Z. Liu, H. Tang, A. Amini, X. Yang, H. Mao, D. Rus, and S. Han, "BEVFusion: Multi-Task Multi-Sensor Fusion with Unified Bird's-Eye View Representation," Jun. 2022, arXiv:2205.13542 [cs].
- [3] H. Caesar, V. Bankiti, A. H. Lang, S. Vora, V. E. Liong, Q. Xu, A. Krishnan, Y. Pan, G. Baldan, and O. Beijbom, "nuScenes: A Multimodal Dataset for Autonomous Driving," in *2020 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. Seattle, WA, USA: IEEE, Jun. 2020, pp. 11 618–11 628.
- [4] F. Ulrich, E. Moser, F. Olbrich, M. Ebert, R. Bierl, and A. Kaup, "Luminance Simulation in CARLA under Cloud Coverage - Model Validation and Implications," in *2023 IEEE International Workshop on Metrology for Automotive (MetroAutomotive)*, Jun. 2023, pp. 228–233.
- [5] F. Ulrich, A. Kaup, and M. Ebert, "Novel Test Bench for End-to-End Validation of Monocular Depth Estimation under the Influence of Glaring Situations," Jul. 2024.
- [6] X. Li, T. Xie, D. Liu, J. Gao, K. Dai, Z. Jiang, L. Zhao, and K. Wang, "Poly-MOT: A Polyhedral Framework For 3D Multi-Object Tracking," Jul. 2023, arXiv:2307.16675 [cs].
- [7] A. Bewley, Z. Ge, L. Ott, F. Ramos, and B. Upcroft, "Simple Online and Realtime Tracking," in *2016 IEEE International Conference on Image Processing (ICIP)*, Sep. 2016, pp. 3464–3468, arXiv:1602.00763 [cs].
- [8] X. Weng, J. Wang, D. Held, and K. Kitani, "3D Multi-Object Tracking: A Baseline and New Evaluation Metrics," Jul. 2020, arXiv:1907.03961 [cs].
- [9] H.-k. Chiu, A. Prioletti, J. Li, and J. Bohg, "Probabilistic 3D Multi-Object Tracking for Autonomous Driving," Jan. 2020, arXiv:2001.05673 [cs].
- [10] J. Gawlikowski, C. R. N. Tassi, M. Ali, J. Lee, M. Humt, J. Feng, A. Kruspe, R. Triebel, P. Jung, R. Roscher, M. Shahzad, W. Yang, R. Bamler, and X. X. Zhu, "A Survey of Uncertainty in Deep Neural Networks," Jan. 2022, arXiv:2107.03342 [cs, stat].
- [11] E. Hüllermeier and W. Waegeman, "Aleatoric and epistemic uncertainty in machine learning: an introduction to concepts and methods," *Machine Learning*, vol. 110, no. 3, pp. 457–506, Mar. 2021.
- [12] A. Kendall and Y. Gal, "What Uncertainties Do We Need in Bayesian Deep Learning for Computer Vision?" in *Advances in Neural Information Processing Systems*, vol. 30. Curran Associates, Inc., 2017.
- [13] V. Kuleshov, N. Fenner, and S. Ermon, "Accurate Uncertainties for Deep Learning Using Calibrated Regression."
- [14] S. Landgraf, M. Hillemann, T. Kapler, and M. Ulrich, "Efficient Multi-task Uncertainties for Joint Semantic Segmentation and Monocular Depth Estimation," Feb. 2024, arXiv:2402.10580 [cs].
- [15] M. Valdenegro-Toro, "Sub-Ensembles for Fast Uncertainty Estimation in Neural Networks," in *2023 IEEE/CVF International Conference on Computer Vision Workshops (ICCVW)*. Paris, France: IEEE, Oct. 2023, pp. 4121–4129.
- [16] D. Feng, L. Rosenbaum, F. Timm, and K. Dietmayer, "Leveraging Heteroscedastic Aleatoric Uncertainties for Robust Real-Time LiDAR 3D Object Detection," *2019 IEEE Intelligent Vehicles Symposium (IV)*, pp. 1280–1287, Jun. 2019, conference Name: 2019 IEEE Intelligent Vehicles Symposium (IV) ISBN: 9781728105604 Place: Paris, France Publisher: IEEE.
- [17] D. Feng, L. Rosenbaum, C. Glaeser, F. Timm, and K. Dietmayer, "Can We Trust You? On Calibration of a Probabilistic Object Detector for Autonomous Driving," *ArXiv*, Sep. 2019.
- [18] D. Levi, L. Gispan, N. Giladi, and E. Fetaya, "Evaluating and Calibrating Uncertainty Prediction in Regression Tasks," Feb. 2020, arXiv:1905.11659 [cs, stat]. [Online]. Available: <http://arxiv.org/abs/1905.11659>
- [19] B. Lakshminarayanan, A. Pritzel, and C. Blundell, "Simple and Scalable Predictive Uncertainty Estimation using Deep Ensembles," 2017.
- [20] K. Bernardin, A. Elbs, and R. Stiefelhofen, "Multiple Object Tracking Performance Metrics and Evaluation in a Smart Room Environment."